CSC3131: DevOps
Report

Ayoub Aldawood

**This was originally developed and first published in the "Newcastle University Computing" GitHub organisation as part of the CSC3131 module. A public copy has then been published on my GitHub here*

Section 1: Introduction

In modern software organisations, DevOps practices are used to shorten the feedback loop between development and operations, reduce deployment risk, and make systems easier to run over time. However, many smaller companies still lack a mature DevOps culture or tooling. This portfolio project for CSC3131 – Development and Operations of Systems addresses that scenario by designing and implementing a small but realistic DevOps-oriented web application and supporting tooling *(CSC3131 Module Team, 2025)*.

The system I developed is a **DevOps Task Tracker**: a simple CRUD web application that allows a team to create, update and delete work items representing DevOps tasks *(Figure 1)*. The root route "/" redirects to "/tasks", which is the main user interface where users interact with the task list. The goal of the application is not to be feature-rich, but to act as a vehicle for experimenting with key DevOps capabilities, including containerisation, automation, deployment to the cloud, logging and monitoring, and continuous integration.

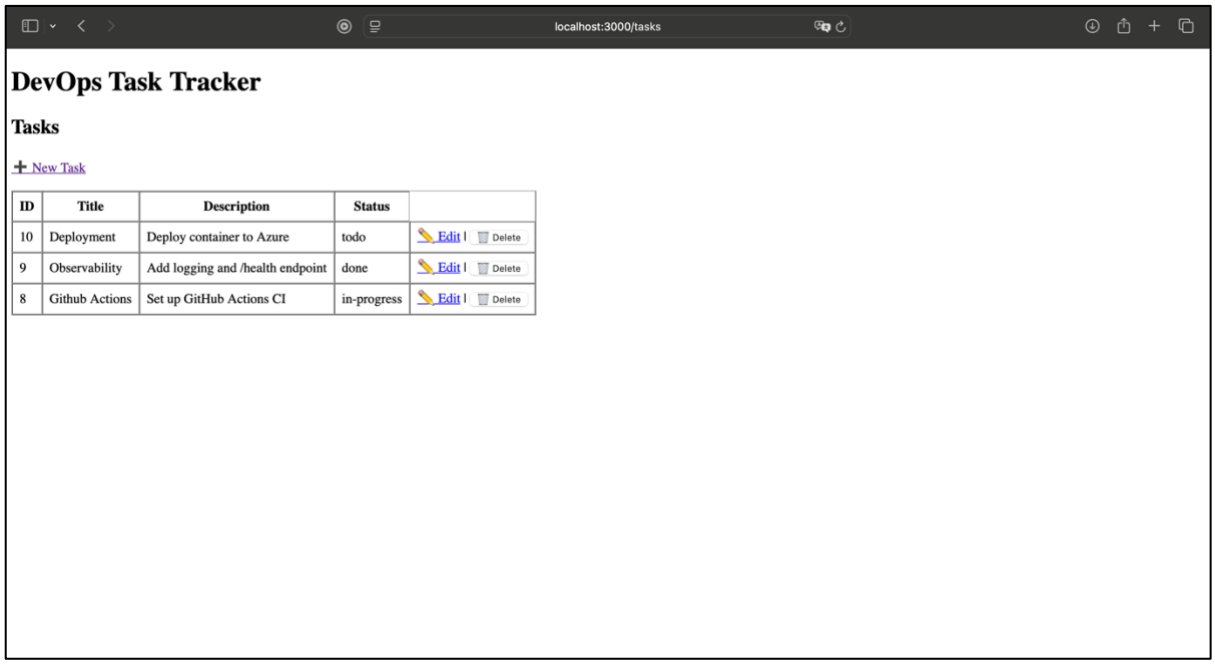

Figure 1: DevOps Task Tracker, showing the CRUD interface where users can create, view, update and delete DevOps work items.

Technically, the application uses a **three-tier architecture**:

- a presentation layer built with **Express** and **EJS** templates for rendering HTML views *(Express.js, 2025)*
- an application layer implemented in **Node.js**, which handles routing, validation and business logic
- a data layer using **SQLite** for persistent storage of tasks *(SQLite, 2025)*.

Around this core, I implemented supporting DevOps tooling. The application is containerised with **Docker** and can be run locally using Docker Compose *(Docker Inc., 2025)*. A container

image is published to **Docker Hub**, and an instance of the system is deployed to **Azure App Service (Linux Web App for Containers)** *(Microsoft, 2025).* Logging is implemented using **Winston**, writing both to the console and to a file (`logs/app.log`), and a simple `/health` endpoint is exposed for automated health checking *(Winston, 2025).* Finally, I configured a **GitHub Actions** workflow so that on each push the repository is checked out, dependencies are installed, automated tests for the `/health` endpoint are run using **Jest and Supertest**, and the Docker image is built *(GitHub, 2025; Jest, 2025; Supertest, 2025).*

This report explains how these tools and practices fit together, and reflects on what they would mean for a small company trying to build its DevOps capabilities. Section 2 provides a high-level overview of the system architecture. Section 3 discusses how each DevOps principle from the assignment brief (continuous integration, deployment, maintainability, scalability, observability and security) is addressed in this project *(CSC3131 Module Team, 2025).* Section 4 focuses on the development process and code quality. Section 5 considers cost and sustainability in the context of Azure. Section 6 evaluates the strengths and limitations of my approach and outlines possible future improvements, before Section 7 concludes the report.

Section 2: System Overview

The "DevOps Task Tracker" follows a simple three-tier architecture, which aligns with the coursework requirement to include a server, a database and a web framework. At the centre of the system is an **Express** application written in **Node.js** *(Express.js, 2025)*. This acts as both the web server and the application layer.

2.1 Presentation layer

The user interface is implemented using server-side rendered **EJS** templates *(Express.js, 2025)*. There are views for listing tasks, creating a new task and editing an existing task. The main `/tasks` page *(Figure 1)* displays a table of tasks with their ID, title, description and status, along with "Edit" and "Delete" actions. A small "New Task" link opens a form where users can submit a new item. Because the UI is rendered server-side, no separate front-end framework is required, which keeps the stack simple and reduces the number of moving parts for this portfolio.

2.2 Application layer

The application logic is organised using Express routers (Express.js, 2025). All task-related routes are defined in `src/routes/tasks.js`, including:

- GET `/tasks` to list tasks,
- GET `/tasks/new` to render the creation form,
- POST `/tasks` to create a new task,
- GET `/tasks/:id/edit` to render the edit form,
- POST `/tasks/:id` to update an existing task,
- POST `/tasks/:id/delete` to delete a task.

Requests are logged through a shared Winston logger, and each create, update and delete operation writes a structured log entry indicating what changed *(Winston, 2025)*. The application also defines a GET `/health` route that returns a small JSON payload indicating that the service is running. This route is used both for local checks and for automated tests.

2.3 Data layer

Persistence is handled by a local **SQLite** database file (`devops_task_tracker.db`) accessed via the `better-sqlite3` library *(SQLite, 2025)*. A small data access module (`src/db.js`) is responsible for creating the tasks table if it does not already exist and for preparing the SQL statements used by the routes. SQLite is sufficient for this small, single-user system, but also makes it easy to demonstrate how a database is wired into the rest of the stack.

2.4 Container and cloud environment

The application can run directly with `npm run dev`, but is also packaged as a Docker image using a `Dockerfile` *(Docker Inc., 2025).* The container image exposes port 3000 and runs `npm start`, and a `docker-compose.yml` file is used to mount the SQLite database and log directory as volumes so that data persists across container restarts *(Docker Inc., 2025)*. In the cloud, the same image is deployed to **Azure App Service for Containers** *(Figure 2),* so the architecture is consistent between local development, Docker and Azure *(Microsoft, 2025)*. This reduces configuration drift and supports DevOps goals such as repeatable deployments and environment parity.

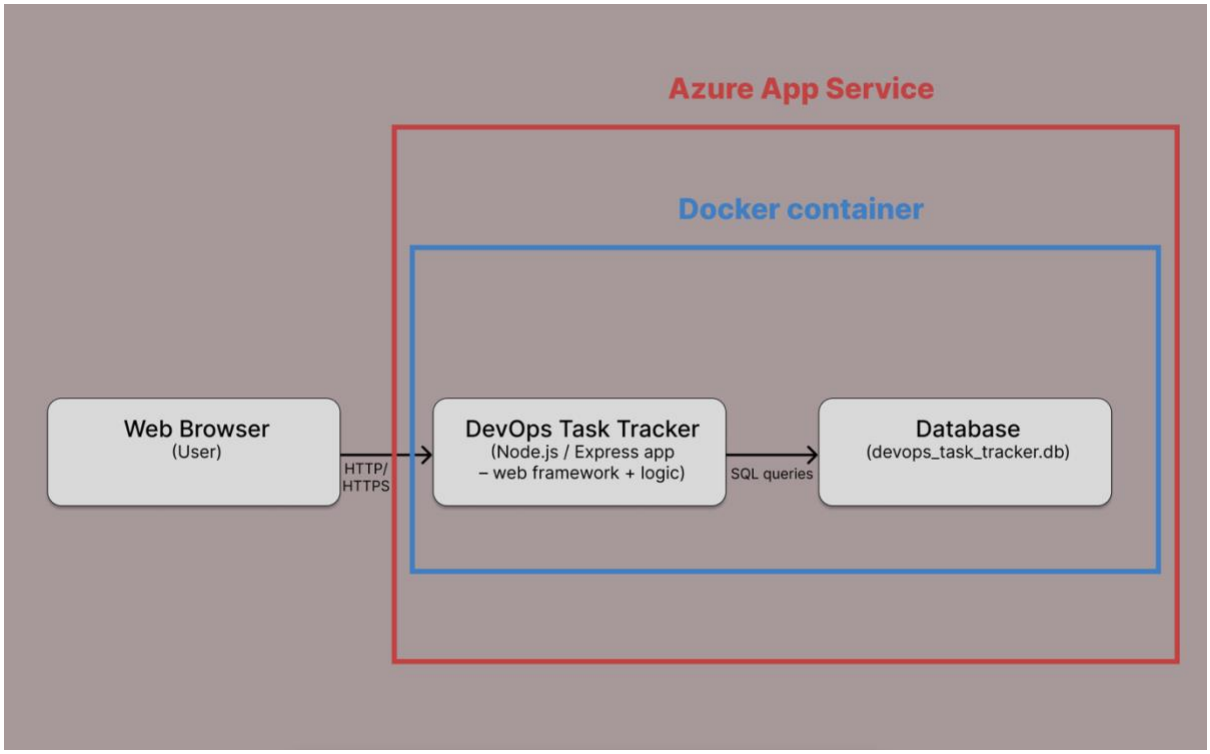

Figure 2: High-level architecture of the DevOps Task Tracker showing the browser client, Node.js/Express application and SQLite database, containerised with Docker and deployed to Azure App Service.

Section 3: DevOps Principles in This Project

The assignment brief highlights several DevOps-related concerns that should be addressed in the portfolio: **continuous integration, deployment, maintainability, scalability, observability and security** *(CSC3131 Module Team, 2025)*. Not all of these need to be fully implemented, but I am expected to understand each of them and explain how they relate to my system. In this section I describe what I have implemented for each principle in the DevOps Task Tracker, and where appropriate I outline how the design could be extended in a more production-ready system.

3.1 Continuous Integration

Continuous Integration (CI) aims to ensure that changes are integrated and validated frequently, so that defects are caught early rather than accumulating until a large release. For this project, I implemented CI using **GitHub Actions** *(GitHub, 2025)*.

The repository contains a workflow file at `.github/workflows/ci.yml`. This workflow is triggered on every push and pull request to the `main` branch. When it runs, it performs a series of automated steps *(Figure 3)*:

- checks out the repository code,
- sets up **Node.js 18** with dependency caching,
- installs dependencies using `npm ci`,
- runs automated tests using `npm test` (Jest + Supertest),
- builds the Docker image using the project's `Dockerfile` *(Jest, 2025; Supertest, 2025; Docker Inc., 2025)*.

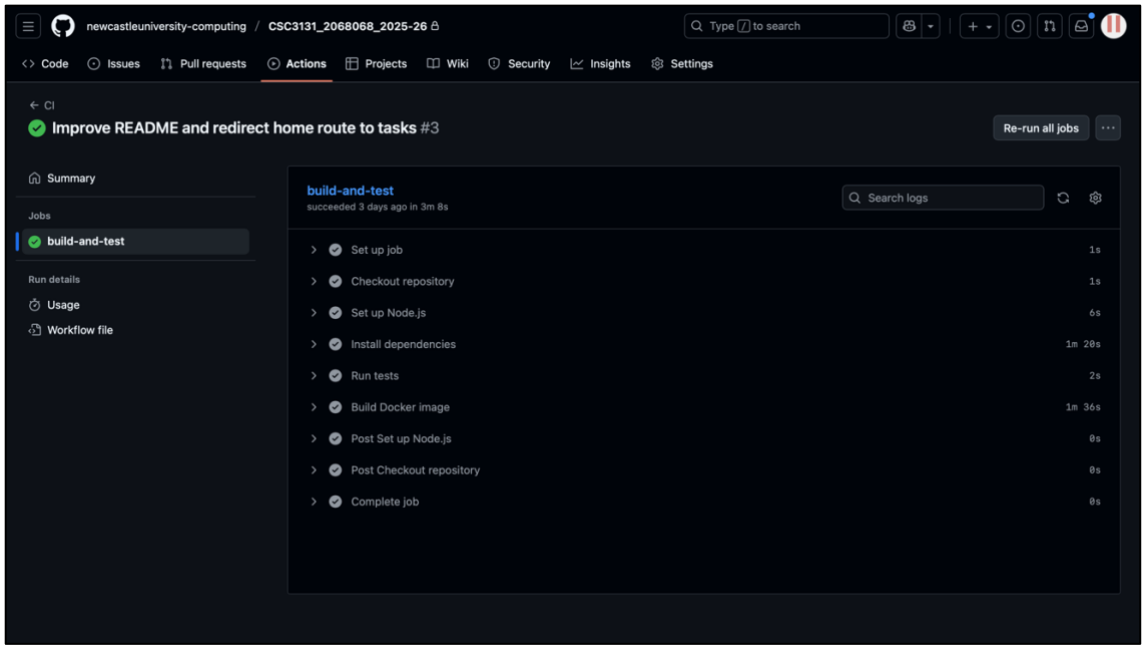

Figure 3: GitHub Actions CI workflow for the DevOps Task Tracker, showing automated steps to install dependencies, run Jest tests and build the Docker image on each push to the main branch.

The test suite currently focuses on the `/health` endpoint, verifying that the service responds with HTTP 200 and a JSON payload containing the expected fields. Although this is a small test, it demonstrates how the application can be automatically exercised on every change and provides a foundation for adding further tests in future.

This CI pipeline means that if I introduce a breaking change (for example, a syntax error, a failing test, or a Dockerfile misconfiguration), the workflow will fail and highlight the issue before any manual deployment to Azure. In a real team, this would support practices such as branch protection rules, where changes can only be merged once the CI pipeline passes.

3.2 Deployment

Deployment in this project is based on **containers** and a simple path from development to the cloud.

Locally, the application can be run directly with `npm run dev`, but it is also packaged into a Docker image using the `Dockerfile`. The container exposes port 3000 and runs `npm start`. A `docker-compose.yml` file is used to orchestrate the container, bind-mounting the SQLite database file (`devops_task_tracker.db`) and the `logs/` directory so that data and logs persist across container restarts *(Docker Inc., 2025)*.

The same image is then pushed to **Docker Hub** under `ayoubaldawood/devops-task-tracker:latest` *(Figure 4)*. In the cloud, I created an **Azure App Service (Linux Web App for Containers)** configured to pull this public image from Docker Hub and expose it over HTTPS *(Figures 5 and 6)*. The Azure Web App uses the same container image as my local Docker setup, which reduces configuration drift between environments and supports the DevOps principle of "build once, run anywhere" *(Docker Inc., 2025; Microsoft, 2025)*.

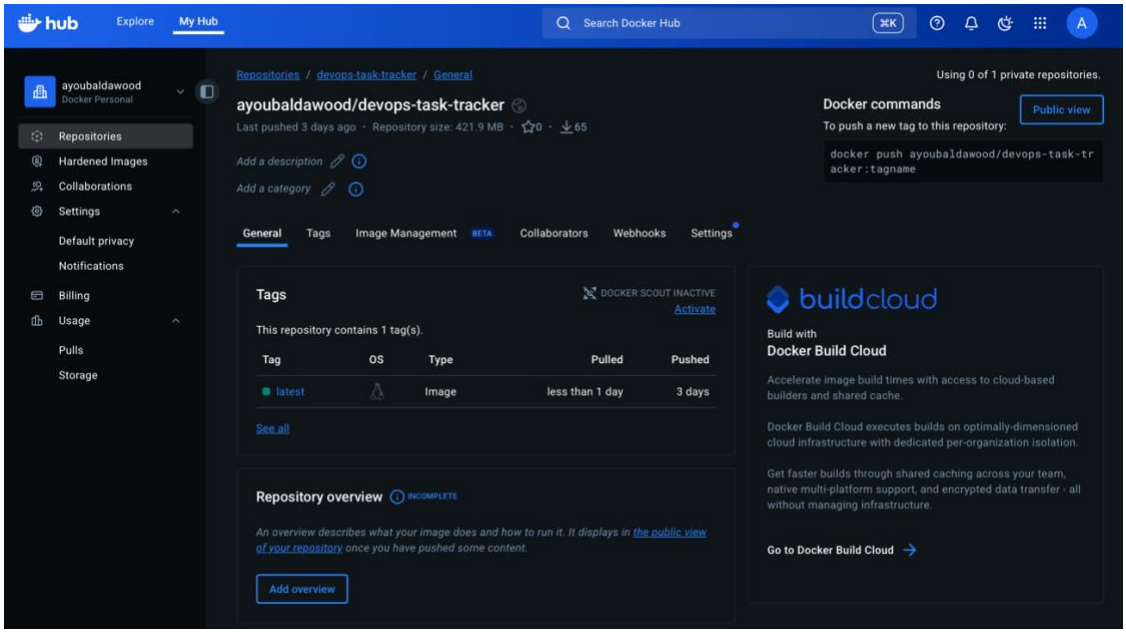

Figure 4: Docker Hub repository for ayoubaldawood/devops-task-tracker, showing the published container image used for deployment to Azure App Service.

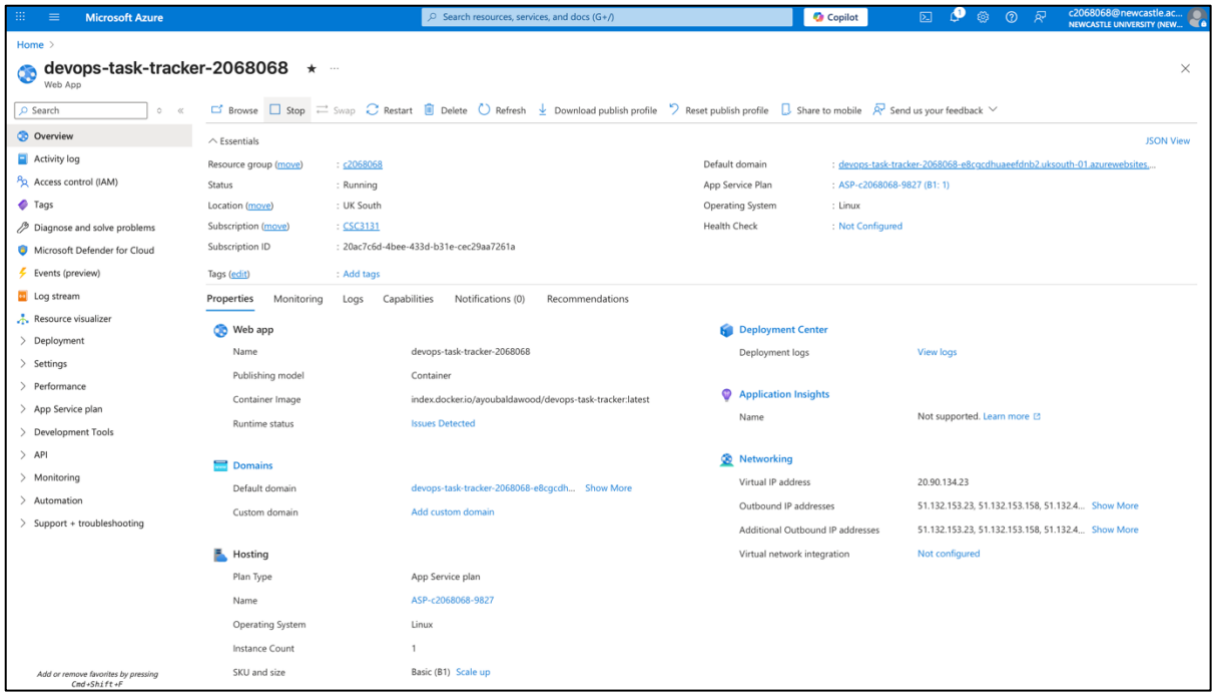

Figure 5: Azure Web App Overview for devops-task-tracker-2068068, showing the Linux Web App for Containers configuration using the Docker image from Docker Hub.

Figure 6: DevOps Task Tracker running on Azure Web App over HTTPS.

At present, deployment to Azure is triggered manually via the Azure Portal, rather than being fully automated from CI. However, the combination of Docker images and the existing CI pipeline means that a continuous deployment or release pipeline could be added relatively easily (for example, by pushing images to Azure Container Registry and updating the Web App configuration automatically). For this coursework, the key point is that the system can be packaged, transported and run consistently across environments using containerisation.

3.3 Observability

Observability covers the ability to understand what the system is doing in production, diagnose problems and monitor its health over time. In this project, I focused on **logging** and a simple **health check**.

Logging is implemented using the **Winston** library in `src/logger.js` *(Winston, 2025)*. The logger is configured with a timestamped, structured log format and two transports:

- **Console output**, which is visible in the terminal for local runs, in Docker logs, and in the Azure App Service log stream.
- A **file transport** that writes to `logs/app.log` *(Figure 7)*. In Docker, this file is bind-mounted from the host (`./logs:/app/logs`) so that logs are available outside the container *(Docker Inc., 2025)*.

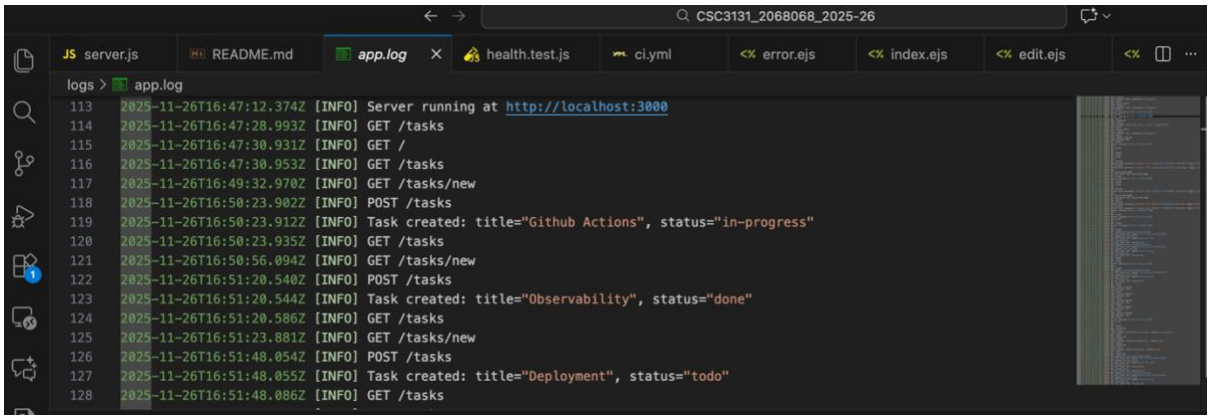

Figure 7: logs/app.log showing timestamped Winston log entries for HTTP requests and task events.

Every HTTP request is recorded via a simple logging middleware in `server.js` that logs the method and URL (for example, `GET /tasks`). In addition, the task routes log higher-level events: creating a task, updating a task and deleting a task each produce a structured log entry with relevant identifiers and status values. Error conditions are also logged: the 404 handler logs “Not Found” events, and the generic error handler records unexpected errors.

The application also exposes a dedicated **health check endpoint** at `/health` *(Figure 8)*. This returns a JSON payload containing a status field (`"ok"`), the process uptime and a timestamp, and is used both by automated tests and by manual inspections. In a more advanced setup, this endpoint could be used by orchestrators or Azure health probes to determine whether the application instance is healthy and should receive traffic *(Microsoft, 2025)*.

Together, these logging and health check mechanisms provide a basic but effective observability layer for a small system, and they align well with the module’s emphasis on logging and monitoring *(CSC3131 Module Team, 2025)*.

Figure 8: /health endpoint returning a JSON payload with service status, uptime and timestamp for monitoring and automated tests.

3.4 Maintainability

Maintainability refers to how easily the system can be understood, modified and extended over time. For this project, I made several design decisions intended to support maintainability, despite the small size of the application.

The codebase is organised into clear modules:

- `src/server.js` handles the application setup, middleware configuration, route registration, health check and error handling.
- `src/routes/tasks.js` encapsulates all task-related HTTP routes and the associated database operations.
- `src/db.js` is responsible for connecting to the SQLite database and initialising the schema.
- `src/logger.js` centralises logging configuration.
- `src/views/` contains the EJS templates for the UI, separated by feature.

This modular structure makes it easier to reason about the system and to change one part (for example, switching database technology) without rewriting everything. The use of environment variables (for instance, `PORT`) also demonstrates basic external configuration rather than hard-coding values *(CSC3131 Module Team, 2025).*

Automated tests, although minimal, contribute to maintainability by providing a safety net when refactoring. The CI pipeline enforces that tests run on every push, reducing the risk of regressions. In addition, a clear README explains how to run, test and deploy the system, which is important for onboarding new developers or future maintainers.

There are still many improvements that could be made, such as adding more comprehensive tests (for the task CRUD routes), introducing a linter (e.g. ESLint) to enforce a consistent code style, and extracting configuration into a dedicated module. However, the current design already demonstrates several maintainability practices appropriate to the scale of the project.

3.5 Scalability

Scalability is about how well the system can handle increased load, either by scaling up (more resources on a single instance) or scaling out (more instances). In this project, I focused mainly on making the application containerised and deployable to a scalable platform, while acknowledging some deliberate limitations.

The **application layer** (Node.js + Express) has been designed to be stateless: requests do not rely on any in-memory global state beyond the database connection. This makes it relatively easy to run multiple copies of the container behind a load balancer, which is how platforms such as Azure App Service achieve horizontal scaling *(Microsoft, 2025)*.

However, the current data layer uses a **local SQLite database file** stored inside or alongside the container. This is convenient for a small coursework project and for demonstrating the server–database interaction, but it is not suitable for a horizontally scaled, multi-instance deployment, because each instance would have its own separate database file *(SQLite, 2025)*.

In a real production setting, to support scalability I would:

- move the data layer to a managed database service such as **Azure SQL Database** or PostgreSQL in Azure,
- configure the application to connect to that external database via environment variables or secrets,
- use Azure App Service's scale-out features to run multiple container instances behind a load balancer *(Microsoft, 2025)*.

This project therefore demonstrates some scalability-friendly practices (containerisation, stateless application layer, deployment to a platform that supports scaling), while using a simplified data storage approach that I discuss explicitly as a limitation.

3.6 Security

Security in this project is intentionally minimal, as the focus is on DevOps practices rather than sensitive data or user accounts. Nevertheless, some security-related considerations are addressed, and others are identified as future work.

On the infrastructure side, deploying to **Azure App Service** means the application is automatically exposed over **HTTPS**, with TLS termination handled by Azure. The use of `.gitignore` ensures that generated artefacts such as `devops_task_tracker.db`, `logs/` and `node_modules/` are not committed to the repository, reducing the risk of accidentally leaking data. Configuration values such as the port are taken from environment variables when available, which is good practice compared to hard-coded values *(CSC3131 Module Team, 2025)*.

At the application level, the current system does not implement authentication or authorisation: any user with the URL can access the task list and modify tasks. For a real internal tool, I would secure the application using one of the mechanisms such as integrating with Azure Active Directory or an OAuth2 provider, and enforcing role-based access controls on sensitive endpoints *(Microsoft, 2025)*. Input validation and more detailed error messages

would also be important to prevent injection attacks and to ensure that operational information is not exposed to end users.

Secrets management is another area which is only partially relevant here (because the system does not use API keys or database passwords), but in a more complex system I would store secrets in **Azure Key Vault** or GitHub Secrets, rather than in the repository or in plain environment files *(Microsoft, 2025; GitHub, 2025)*.

Overall, the current implementation is appropriate for a self-contained coursework project, but in the report I highlight how the same DevOps-style tooling: containers, CI, logging and cloud hosting, could be layered with stronger security controls in a production context.

Section 4: Development Process and Code Quality

During development I tried to treat this small application as a "real" project. All work was done in a Git repository in the Newcastle University GitHub organisation, with frequent, focused commits such as "Add Dockerfile and docker-compose", "Implement logging with Winston" and "Configure GitHub Actions CI workflow" *(GitHub, 2025; Winston, 2025)*. This commit history shows how the system evolved from a basic Express app into a containerised, observable service with CI.

For each change, I followed a simple loop: implement the feature in VS Code, run it locally with `npm run dev`, then test it again in Docker using `docker compose up --build`. Once it worked in both environments I pushed to GitHub so that the CI workflow could rerun the Jest tests and Docker build on a clean runner *(Docker Inc., 2025; Jest, 2025)*. This helped catch mistakes such as missing dependencies or mis-configured scripts.

To support code quality and handover, I kept the project structure small and consistent and documented it in `README.md` and `REQUIREMENTS.md`. Together these files explain how to run, test and deploy the system and where each DevOps feature lives in the codebase, so another teammate could realistically continue the work *(CSC3131 Module Team, 2025)*.

Section 5: Cost and Sustainability in the Cloud

Because the DevOps Task Tracker runs on Azure, I also had to think about cost and sustainability, even though this is only a coursework project *(CSC3131 Module Team, 2025)*.

I deployed the Docker image to an Azure App Service (Linux Web App for Containers) using a small **Basic (B1)** plan. This tier gives enough CPU and memory for a single Node.js container, but is still relatively cheap *(Microsoft, 2025)*. Following the module guidance, I routinely **stopped the Web App** in the Azure Portal when I was not testing. This simple habit avoids paying for a container that is sitting idle and reduces unnecessary use of cloud resources *(CSC3131 Module Team, 2025)*.

Running in the cloud has pros and cons from a sustainability point of view. A shared Azure data centre can use hardware more efficiently than a single under-used on-premise server, but it is also very easy to leave services running and quietly wasting energy *(Microsoft, 2025)*. Part of "doing DevOps well" is therefore keeping track of what is deployed and shutting things down when they are no longer needed.

If this were a real production system, I would go further by describing the Azure resources in **infrastructure-as-code**, adding basic monitoring and alerts for cost, and using autoscaling rules so extra capacity is only added when traffic actually increases *(Microsoft, 2025)*. For the purposes of this project, choosing a small plan and turning the app off when idle were realistic first steps.

Section 6: Evaluation and Reflection

This section evaluates the DevOps Task Tracker against the goals of the coursework and reflects on what I learned during the project.

Overall, I am satisfied that the system demonstrates a realistic slice of DevOps practice around a small web application. I implemented a simple but complete CRUD system with a three-tier architecture, then wrapped it with containerisation, cloud deployment, logging, a health check and a working CI pipeline *(Docker Inc., 2025; Microsoft, 2025; Winston, 2025; GitHub, 2025)*. This aligns well with the assignment emphasis on "using a system as a vehicle" for DevOps tools, rather than building a large feature set *(CSC3131 Module Team, 2025)*.

Several aspects went particularly well. Containerising the application with Docker and using the same image locally and in Azure helped me appreciate the value of consistent environments *(Docker Inc., 2025; Microsoft, 2025)*. Implementing structured logging and a `/health` endpoint showed how relatively small code changes can significantly improve observability *(Winston, 2025)*. Setting up GitHub Actions to run Jest tests and build the Docker image on every push made the development process feel more professional, and the initial failed runs I faced were a useful reminder that CI pipelines need iteration and debugging like any other code *(GitHub, 2025; Jest, 2025)*.

There are also clear limitations. The system uses a local SQLite database file, which is appropriate for a single-instance coursework project but not for a scalable, multi-instance deployment *(SQLite, 2025)*. The test suite is deliberately small and focuses only on the health endpoint, so it does not provide strong regression protection for the task routes. Security is minimal: there is no authentication or authorisation, and no explicit input validation beyond what Express and SQLite provide. These were conscious trade-offs to keep the scope manageable, but they are important to acknowledge.

If I were to continue this project beyond the module, my priorities would be to move the data layer to a managed database service, strengthen the test suite (especially for task creation and editing), and introduce a basic authentication layer, possibly integrating with Azure Active Directory *(Microsoft, 2025)*. I would also look at expressing the Azure infrastructure using templates so that environments could be recreated reliably.

In summary, the project has given me practical experience of how DevOps ideas (CI, containers, logging, cloud hosting and cost awareness) fit together around a real codebase. It has also highlighted that DevOps is an ongoing process of improvement rather than a fixed checklist, even a small system like this can always be made more secure, more testable and more automated.

Section 7: Conclusion

This portfolio set out to build a small but realistic system that could act as a vehicle for exploring DevOps practices. The DevOps Task Tracker provides a complete, three-tier CRUD web application using Node.js, Express, EJS and SQLite, and I used it as a foundation to experiment with containerisation, cloud deployment, logging, health checks and automation *(Express.js, 2025; SQLite, 2025; Docker Inc., 2025; Microsoft, 2025; Winston, 2025)*.

By packaging the system into a Docker image, deploying it to Azure App Service for Containers, and wiring in logging and a `/health` endpoint, I was able to see how development and operations concerns connect around a real codebase. Introducing a GitHub Actions CI pipeline, even with a minimal test suite, also made the workflow feel more robust and professional *(GitHub, 2025; Jest, 2025)*.

The project is intentionally small and has clear limitations, but it has given me practical experience with core DevOps ideas and a clearer understanding of how I would extend this approach in a larger, production system.

References:

- Microsoft. (2025). *Azure App Service documentation*. Available at: https://learn.microsoft.com/azure/app-service/
- Microsoft. (2025). *Azure App Service for containers*. Available at: https://learn.microsoft.com/azure/app-service/overview
- Docker Inc. (2025). *Docker Overview*. Available at: https://docs.docker.com/get-started/overview/
- Docker Inc. (2025). *Docker Compose documentation*. Available at: https://docs.docker.com/compose/
- GitHub. (2025). *GitHub Actions documentation*. Available at: https://docs.github.com/actions
- Express.js. (2025). *Express Guide*. Available at: https://expressjs.com/
- SQLite. (2025). *SQLite Documentation*. Available at: https://sqlite.org/docs.html
- Jest. (2025). *Jest – JavaScript Testing Framework*. Available at: https://jestjs.io/docs/getting-started
- Supertest. (2025). *Supertest – Super-agent driven library for testing HTTP servers*. Available at: https://github.com/ladjs/supertest
- Winston. (2025). *Winston logger documentation*. Available at: https://github.com/winstonjs/winston
- CSC3131 Module Team (2025). *Development and Operations of Systems: Lecture Notes and Practical Sheets*. Newcastle University.